Git and CI/CD compact cheatsheet

STEP 1 Getting started with Git

1.1 What Git is and how to set it up

Version control Records every version of your files with author, time and message, so you can compare, restore and work in parallel.

Distributed Every Git clone has the full history. Commit, log and diff work offline. Centralised tools need the server.

Snapshots A commit is a snapshot of the whole project. Unchanged files are reused, not stored again.

Config levels system, global (~/.gitconfig), local (.git/config). Later wins. Set user.name, user.email and init.defaultBranch first.

Git is not GitHub Git is the tool. GitHub, GitLab and Bitbucket host repositories and add pull requests, reviews and CI/CD.

Set a value for your user
```
git config --global <key> "<value>"
```

Which value wins system < global < local

TRAP Running git init in the wrong folder, for example your whole home folder, makes Git track everything. If that happens, delete the new .git folder in that place. Your files are not touched.

EXAM Git and GitHub are not the same thing. Git is the tool on your computer. GitHub, GitLab and Bitbucket are websites that host Git repositories and add pull requests, issues and CI/CD pipelines.

1.2 The three areas: working tree, staging area and repository

Three areas Working tree (your files), staging area or index (next snapshot), repository (all commits). add moves changes to the index, commit saves the index.

Status codes git status -s prints XY: X staged (index vs HEAD), Y unstaged (folder vs index). ?? untracked, MM staged and changed again.

Commit saves the index Edits made after git add are not in the commit until you add again. commit -a stages tracked files only.

Partial staging git add -p stages hunk by hunk so one file can feed two clean commits.

Good messages Short imperative summary near 50 characters, blank line, then why. One logical change per commit.

Short status
```
git status -s → XY path
```

Stage
```
git add <file>, git add ., git add -p
```

Commit
```
git commit -m "message"
```

TRAP git commit saves what is staged, not what is in your file. If you edit a file after git add, the newer edit is not in the commit until you add it again.

1.3 Reading history: log, diff, blame and naming commits

git log --oneline --graph --all for the picture. -S text to find when code appeared. -p and --stat for what changed. -- path for one file.

Three diffs git diff: folder vs index. git diff --staged: index vs HEAD. git diff HEAD: folder vs HEAD.

Hunks @@ -a,b +c,d @@: old lines a to a+b-1 became new lines c to c+d-1. A changed line is one - and one +.

Naming commits X~n: n first parents back. X^n: n-th parent of a merge. HEAD@{n}: where HEAD was n moves ago.

Ranges A..B: in B not in A. A...B: in one but not both (log), or merge base to B (diff).

Compact history
```
git log --oneline --graph --decorate --all
```

Find code
```
git log -S "price_limit"
```

Hunk header `@@ -a,b +c,d @@`

Ancestors
X~n = n first parents back. X^n = the n-th parent of X

Two dots
A..B = reachable from B but not from A

TRAP git diff with no arguments shows nothing after git add. The changes did not vanish: they moved to the index. Use git diff --staged.

EXAM HEAD~2 and HEAD^^ name the same commit. HEAD^2 is different: it is the second parent of HEAD and only exists if HEAD is a merge commit.